KPRIT

Imagine. Innovate. Inspire.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

PROJECT

PRESENTATION

atch-Level Texture Classification in Remote Sensing Using Transfer Learning on TILDA-40

PRESENTED BY



SYEDA SANOBAR ALI(22RA1A6673)



P.SHIVA MUKESH(22RA1A6676)



A.ABHILASH(23RA5A6601)

UNDER THE GUIDANCE OF
MRS G.SUJATHA
ASSISTANT PROFESSOR

Abstract

- Remote sensing uses texture analysis for tasks like environmental monitoring, agriculture, urban growth, and resource management. Traditional methods like GLCM are simple and fast but struggle when image resolution, lighting, or terrain changes.

- This study proposes a transfer learning approach using pretrained deep learning models on the TILDA-400 dataset to classify image textures. The process includes preprocessing, patch extraction, augmentation, feature transfer, and classification.

- The proposed method improves accuracy, efficiency, and robustness compared to traditional techniques, making it suitable for large-scale and real-time remote sensing applications.

Introduction

- Fabric defect detection is important for maintaining textile quality, but most factories still rely on human inspection, which is tiring and only about 60–75% accurate. Automating this process is challenging due to continuous production, many types of defects, and different quality standards.

- Traditional computer vision methods were not very reliable in real factory conditions. Deep learning methods perform better, but they require large and well-labeled datasets. However, unclear defect definitions, rare defects, subtle differences, and texture variations create data imbalance, making supervised learning less effective.

Research Objectives

- Develop a transfer-learning framework for patch-level texture classification using the TILDA-400 dataset.
- Fine-tune selected models for discriminative texture learning with minimal data and training time.
- Assess performance using accuracy, precision, recall, F1-score, and confusion matrix.
- Validate computational efficiency, robustness, and scalability under varying texture and environmental conditions.

Traditional System

The traditional method depends on a trained person visually checking fabric or image patches one by one.

- The traditional method relies on human visual inspection.
- A trained analyst checks fabric or image patches one by one.
- Images may be printed and manually divided into smaller patches.
- Each patch is compared with reference samples or texture atlases.
- Classification is based on visible features like texture size, uniformity, pattern, direction, and brightness.
- The process is subjective and depends on the analyst's experience.
- Finally, the defect or texture type is manually labeled and recorded.

Problem Statement

- Traditional texture and defect detection depends on manual inspection, which is slow and labor-intensive.
- Human judgment can be subjective and affected by fatigue or experience level.
- Handcrafted feature methods cannot capture complex texture patterns effectively.
- Existing systems perform poorly under changes in lighting, scale, or environmental conditions.
- Manual approaches are not suitable for large-scale or real-time applications.
- Handling large, complex, and unbalanced data is difficult with traditional methods.
- Therefore, a robust, automated, and scalable system with higher accuracy is needed.

Significance – Need of AI

- AI reduces human errors, fatigue, and subjectivity in inspection .
- Deep learning automatically learns complex texture patterns better than manual features.
- AI systems provide higher accuracy and work well under different lighting and environmental conditions.
- Transfer learning helps build effective models even with limited or unbalanced data.
- AI-based systems are scalable and suitable for real-time, large-scale applications.

LITERATURE SURVEY

S. No.	Author & Year	Method / Technique	Dataset / Domain	Key Findings	Limitations
1	Zhang et al., 2023	VGG19-based Texture Feature Extraction	Industrial texture images	Deep features outperform handcrafted features	High computational cost
2	Li et al., 2023	CNN Features + SVM Classifier	Surface defect datasets	Improved accuracy over traditional ML	Sensitive to kernel parameters
3	Kumar et al., 2023	VGG16/VGG19 + LDA	Texture classification	Effective dimensionality reduction	Linear separability assumption
4	Chen et al., 2023	Deep Features + QDA	Industrial inspection	Handles class variance better than LDA	Poor performance with very small datasets
5	Singh et al., 2023	CNN Feature Extraction + SVM	Fabric defect detection	Robust classification with limited data	Multi-stage pipeline complexity
6	Park et al., 2023	VGG19 + Classical ML Models	Remote sensing textures	Deep features enhance ML classifiers	High memory usage
7	Rahman et al., 2024	VGG19 Feature Extraction + LDA	Textile inspection	Stable results with small datasets	Limited non-linear modeling
8	Liu et al., 2024	CNN-based Feature Learning + QDA	Surface texture analysis	Improved class separation	Sensitive to noise

9	Ahmed et al., 2024	Deep Features + SVM (RBF Kernel)	Industrial texture datasets	High classification accuracy	Requires careful hyperparameter tuning
10	Zhou et al., 2024	VGG19 + Extra Trees Classifier (ETC)	Texture classification	Fast training and robust ensemble learning	Large model size
11	Patel et al., 2024	CNN Feature Extraction + ETC	Manufacturing defect detection	Handles feature variability effectively	Reduced interpretability
12	Kim et al., 2024	Transfer Learning using VGG19	Texture recognition	Good generalization with limited data	Computationally intensive
13	Rao et al., 2025	VGG19 + SVM	TILDA-like remote sensing dataset	Accurate patch-level classification	High training time
14	Sharma et al., 2025	Deep Features + LDA and QDA	Industrial texture datasets	Simple and interpretable classifiers	Lower accuracy for complex textures
15	**Al-Mansoori et al., 2025**	**VGG19 Feature Extraction + Extra Trees Classifier**	**Fabric defect detection**	**High accuracy with reduced training time**	**Performance depends on feature quality**

Research Gap

Although VGG19 combined with classifiers like LDA, QDA, and SVM improves texture and defect classification accuracy, several challenges still remain. Many studies evaluate classifiers separately instead of using a unified framework for fair comparison. Existing approaches also struggle with limited labeled data, class imbalance, and variations in texture appearance. In addition, computational efficiency and real-time performance are often not adequately addressed. Therefore, there is a need for an efficient and scalable transfer learning framework that integrates VGG19 with multiple classifiers for more robust and practical texture classification.

Proposed System – Patch-Level Texture Classification

The proposed system uses transfer learning with VGG19 to classify texture patches from the TILDA-400 dataset. It automatically extracts deep texture features and uses the Extra Trees Classifier for accurate and fast prediction.The workflow includes patch extraction, preprocessing, feature extraction, and classification. It reduces manual effort and works well even with limited data. The system is robust to noise, lighting changes, and texture variations. Overall, it offers better accuracy and scalability for real-time remote sensing applications.

System Architecture – Patch-Level Texture Classification (TILDA-400)

The proposed system follows a **transfer learning–based architecture** for patch-level texture classification using deep features and machine-learning classifiers.

1. TILDA-400 Dataset Input

- The process starts with the **TILDA-400 remote sensing dataset**.
- Images contain multiple texture categories relevant to remote sensing and defect detection.

2. Image Preprocessing

- Input images are resized to **224 × 224 pixels** to match the requirements of pretrained CNN models.
- Noise reduction and normalization help enhance texture patterns.

3. VGG19 Feature Extraction

- A **pretrained VGG19 Convolutional Neural Network** is used as a **feature extractor**.
- Fully connected layers of VGG19 are removed.
- Deep convolutional features representing **multi-scale texture information** are extracted.

4. Feature Dataset Preparation

- Store extracted VGG19 feature vectors with labels to avoid recomputation

5. Model Framework

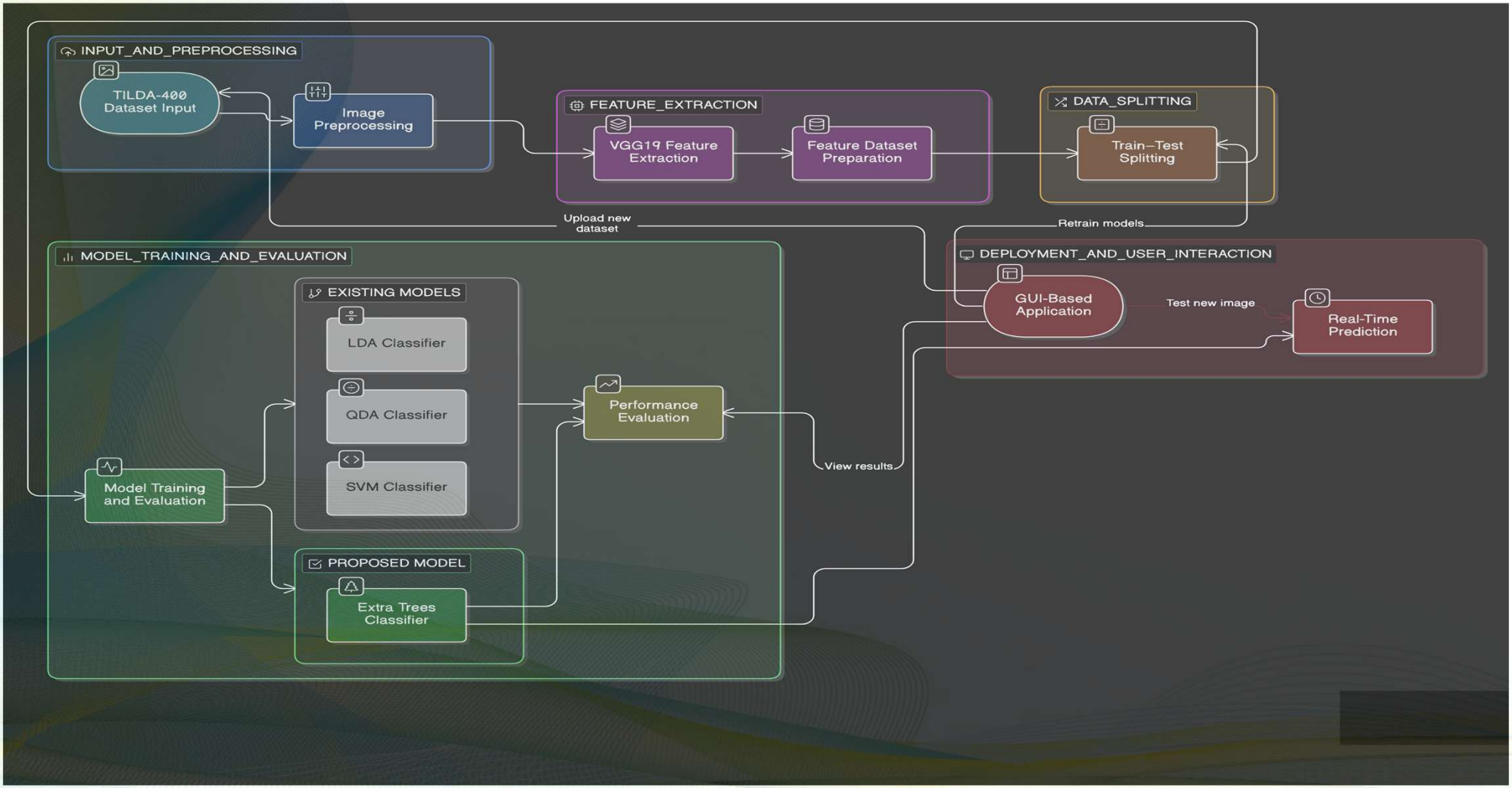
- Compare baseline models (LDA, QDA, SVM) with the proposed **Extra Trees Classifier (ETC)**.
- ETC selected for faster training, better feature handling, and higher accuracy.

6. Performance Evaluation

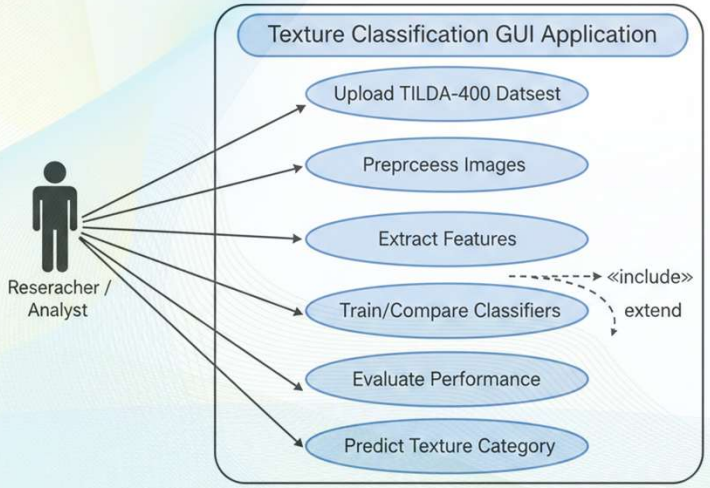
- Evaluated using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC/AUC.

7. GUI Application

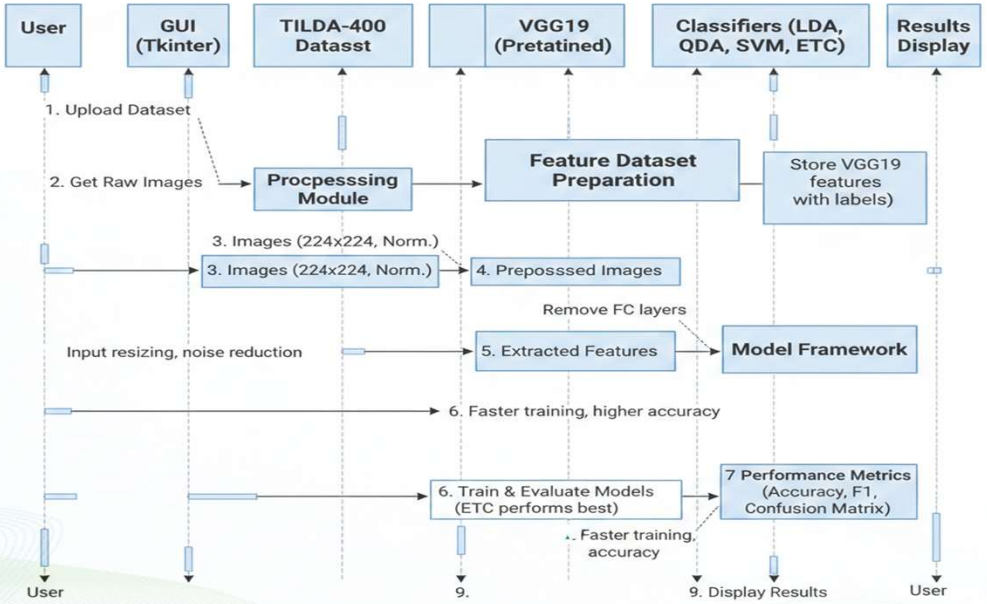
- Tkinter-based end-to-end system for dataset upload, training, testing, and result visualization.


INPUT_AND_PREPROCESSING
TILDA-400 Dataset Input
Image Preprocessing
FEATURE_EXTRACTION
VGG19 Feature Extraction
Feature Dataset Preparation
DATA_SPLITTING
Train–Test Splitting
Upload new dataset
Retrain models
MODEL_TRAINING_AND_EVALUATION
EXISTING MODELS
LDA Classifier
QDA Classifier
SVM Classifier
Model Training and Evaluation
Performance Evaluation
PROPOSED MODEL
Extra Trees Classifier
View results
DEPLOYMENT_AND_USER_INTERACTION
GUI-Based Application
Test new image
Real-Time Prediction

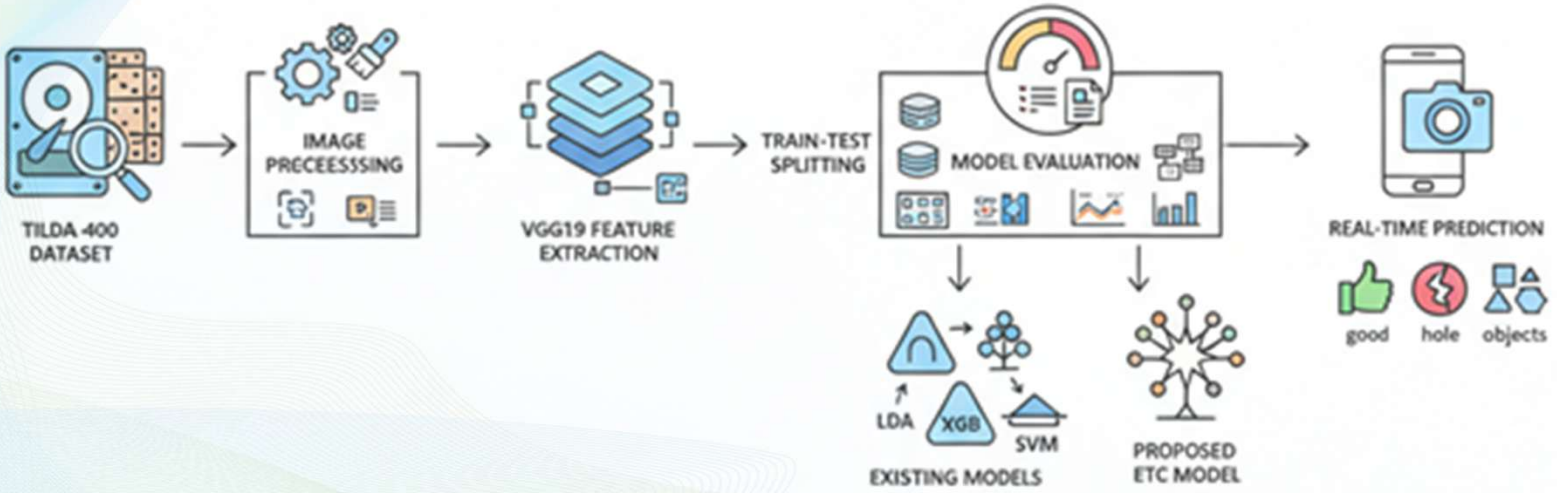
Use case diagram



System Sequence Diagram



System Data Flow



DATASET

Dataset Name: TILDA-400
Dataset Type: Image-based texture dataset
Data Format: Grayscale images organized in class-wise folders

Input (Independent Variables)
- Raw texture images
- Pixel intensity values
- No handcrafted features used

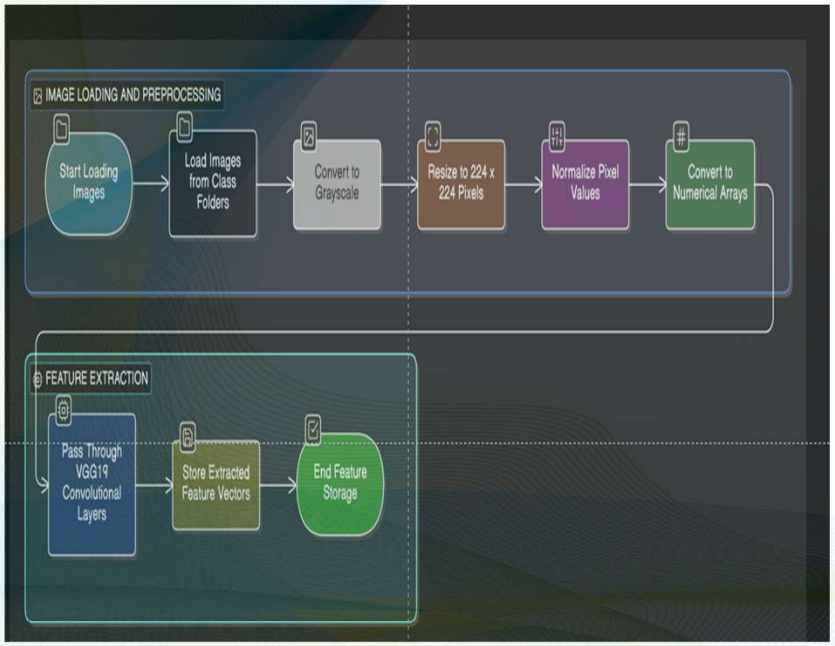
Target (Dependent Variable)
- **Column Name:** Target_Label
- Labels derived from folder names

Feature Generation
- All images resized to **224 × 224 pixels**
- Preprocessing ensures uniform input size

Dataset Split
- Dataset randomly shuffled
- Divided into training and testing sets

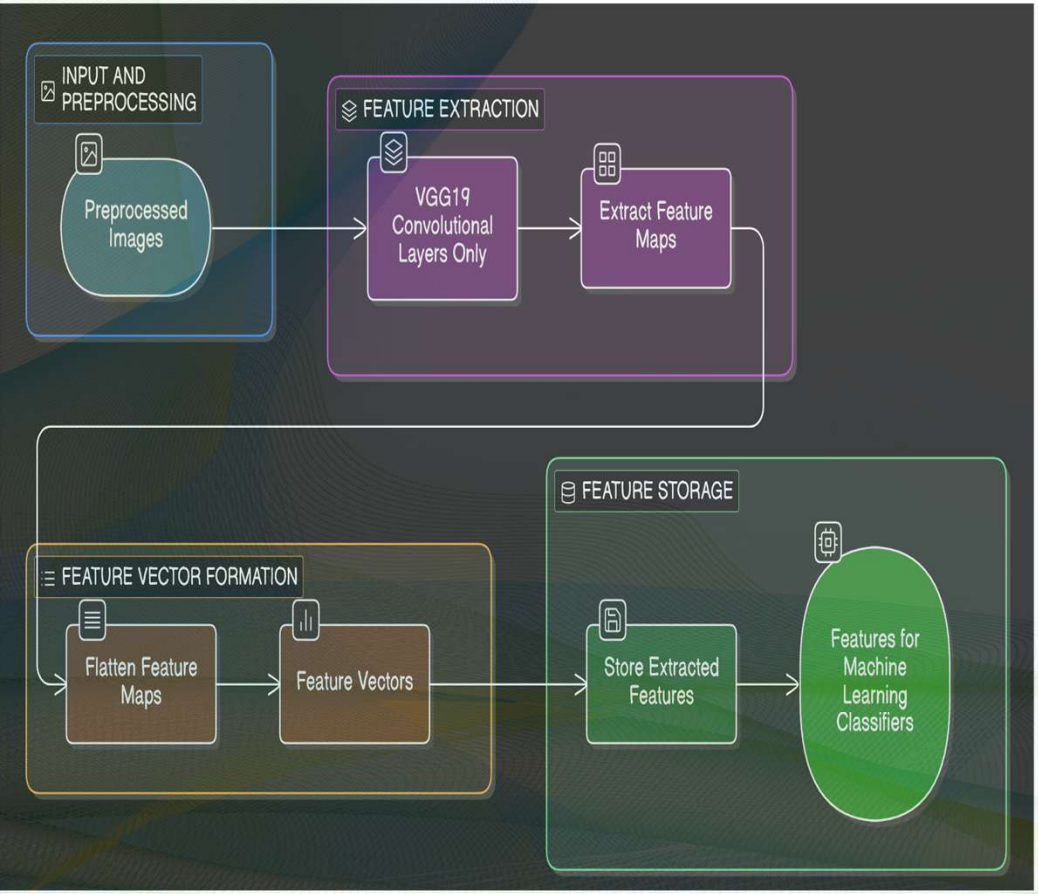
Column Name	Description
Image_ID	Unique identifier assigned to each image
Image_Data	Raw grayscale pixel values of the texture image
Height	Height of the image in pixels
Width	Width of the image in pixels
Extracted_Features	Deep feature vector obtained using VGG19
Target_Label	Texture class of the image

Image Preprocessing Steps



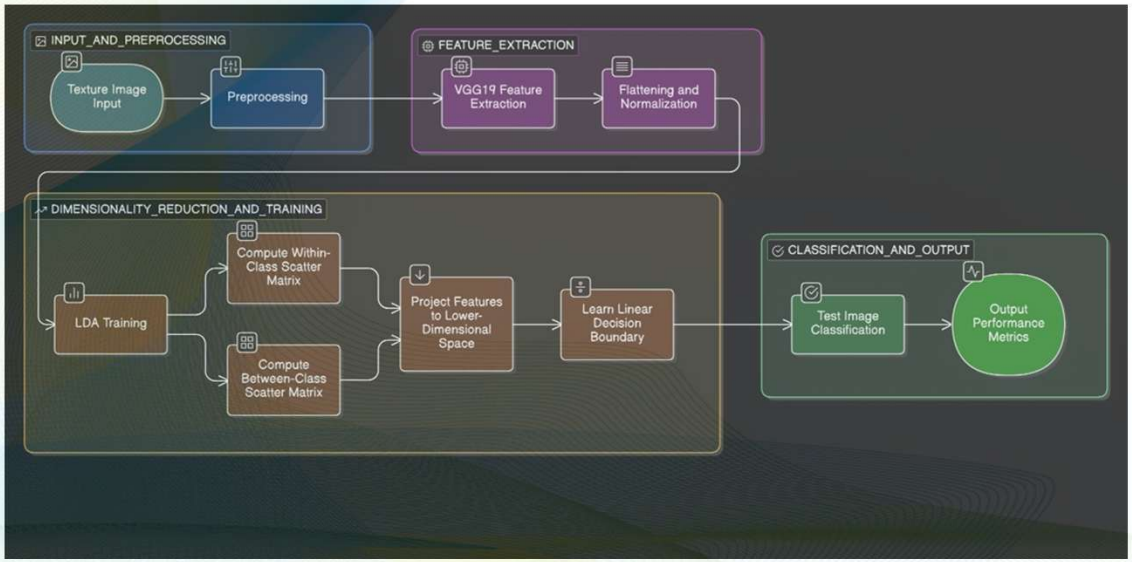
- Texture images are loaded from class-wise folders and converted into grayscale format.
- All images are resized to **224 × 224 pixels** to match the input requirements of the pretrained **VGG19** model.
- Pixel values are normalized to reduce illumination variations and improve model stability.
- Images are converted into numerical arrays compatible with deep learning frameworks.

Feature Extraction Steps



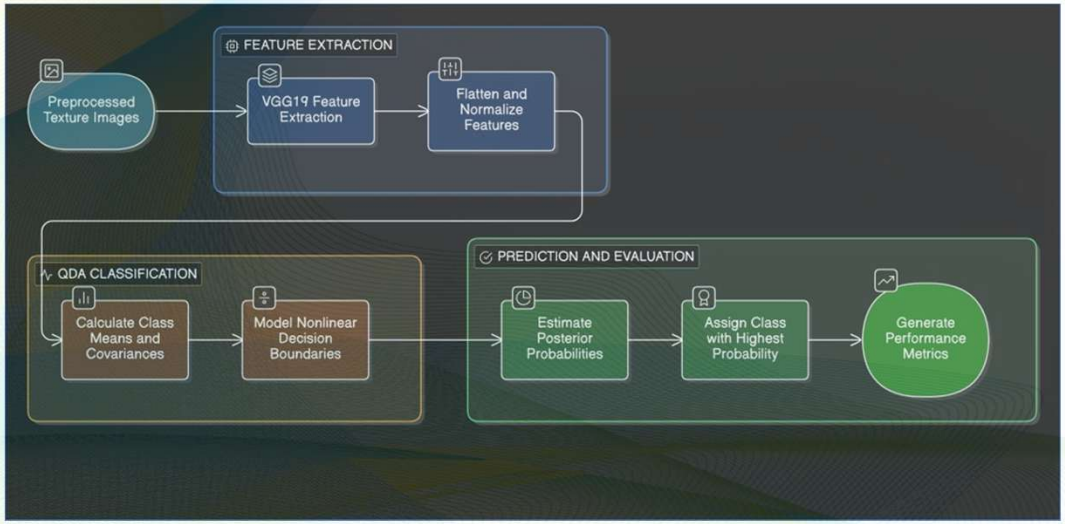
- Preprocessed images are given as input to a pretrained **VGG19** model.
- Fully connected layers are removed; only convolutional layers are used.
- Deep layers extract high-level, multi-scale texture features.
- Output feature maps are flattened into fixed-length numerical vectors.
- These vectors represent important texture characteristics.

Existing Algorithm 1: LDA with VGG19 Features



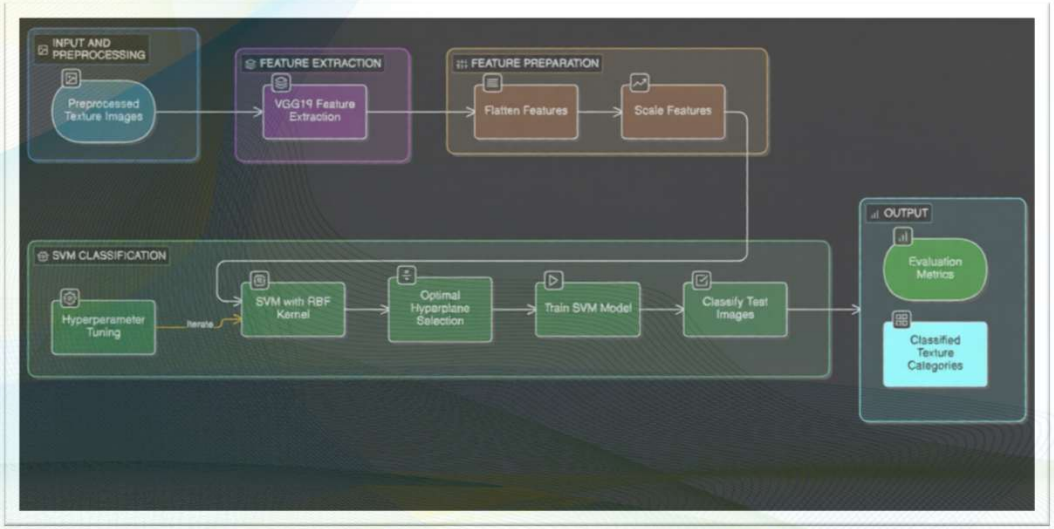
- Preprocessed images are passed through pretrained VGG19 to extract deep feature vectors. Extracted features are flattened and normalized before classification.
- The feature vectors are given as input to the LDA classifier. LDA calculates within-class and between-class scatter matrices.

Existing Algorithm 2: QDA with VGG19 Features



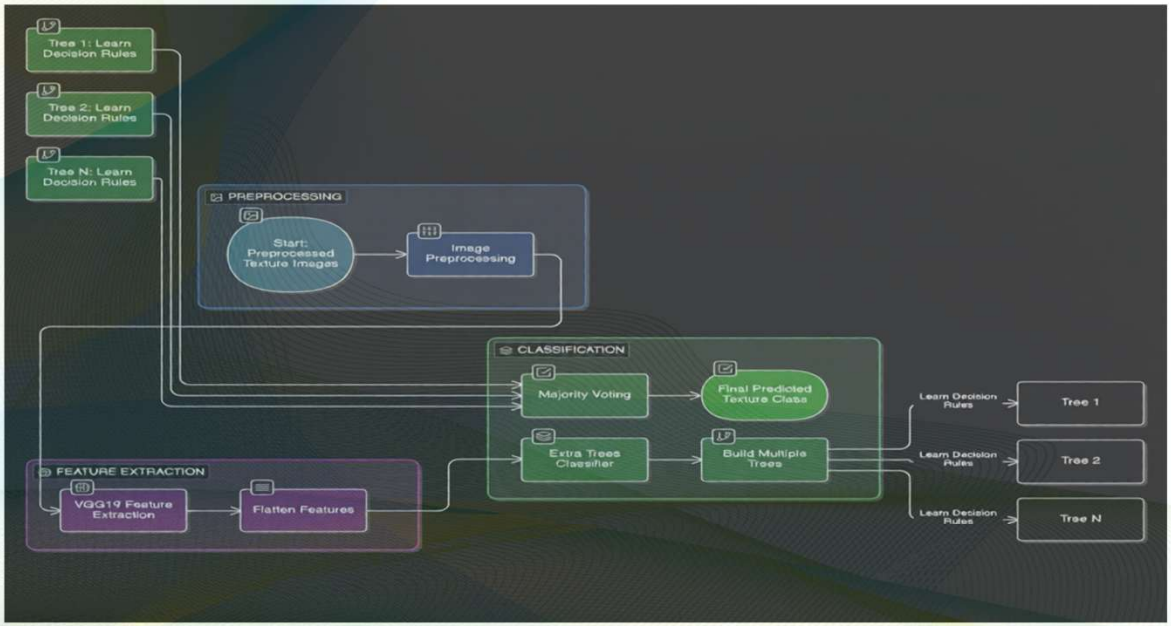
- Preprocessed images are passed through pretrained **VGG19** to extract deep feature vectors.
- Extracted features are flattened and normalized before classification.
- The feature vectors are provided to the **QDA** classifier.
- QDA computes class-specific mean vectors and covariance matrices.
- It forms non-linear decision boundaries using quadratic discriminant functions

Existing Algorithm 3: SVM with VGG19 Features



- Preprocessed images are passed through pretrained **VGG19** to extract deep feature vectors.
- Features are flattened and scaled before classification.
- The processed features are given to an **SVM** classifier.
- An **RBF kernel** maps features into a higher-dimensional space.
- SVM finds the optimal hyperplane by maximizing the margin between classes.
- Hyperparameters (C and gamma) are tuned for better performance.

Proposed Algorithm: ETC with VGG19 Features



•. Preprocessed images are passed through pretrained **VGG19** to extract deep texture feature vectors.

•Extracted features are flattened and given as input to the **Extra Trees Classifier (ETC)**.

•ETC builds multiple decision trees using random feature selection and random splits.

•Each tree learns classification rules independently from deep features.

System Requirements

Software Components

- Python: 3.7.6
- Operating system: Windows
- Frontend: Tkinter
- Database: Redis

Hardware Components

- Processor: Intel i5
- Memory: 1 TB (storage)
- RAM

Python Packages

1. GUI & Basic

- tkinter
- pickle

2. Data Processing

- pandas
- numpy

3. Visualization

- matplotlib
- seaborn

4. Machine Learning

- scikit-learn
- scikit-image

5. Deep Learning

- joblib
- scipy

Frontend Environment

- The frontend of the system is developed using Python Tkinter, which provides a lightweight and user-friendly graphical interface.
- Tkinter is used to design interactive components such as buttons, labels, image display panels, and text fields.
- The interface allows users to upload datasets, select images, initiate model training, and perform real-time texture prediction.
- Event-driven programming is used to connect user actions with backend processing modules.
- The frontend integrates seamlessly with deep learning and machine learning models, enabling end-to-end interaction within a single desktop application.

Database Integration

Purpose: Fast and efficient storage of extracted features and model-related data for large datasets.
Database: Redis (high-speed, in-memory key–value store).
Stores:
VGG19 extracted feature vectors
Patch labels (good, hole, objects, etc.)
Train–test split indices
Model metadata and configurations
Benefits:
Avoids repeated feature extraction
Reduces computation time
Speeds up experimentation and evaluation
Supports scalability for large-scale remote sensing tasks
Enables quick model reuse and version tracking

VGG19 Feature Extractor Sample code

```python
import numpy as np
from tensorflow.keras.applications import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input
from PIL import Image
import os

base_model = VGG19(weights='imagenet', include_top=False, pooling='avg')
categories = ['cat', 'dog']  # your categories
X, y = [], []

for i, cat in enumerate(categories):
    for filename in os.listdir(f'./{cat}'):  # your folders
        if filename.endswith('.jpg'):
            img = Image.open(f'./{cat}/{filename}')
            img = np.array(img.resize((224,224)))
            img = preprocess_input(img[np.newaxis,...])
            feat = base_model.predict(img, verbose=0).flatten()
            X.append(feat); y.append(i)

print(f"Features shape: {np.array(X).shape}")
```

Sample Code for LDA+VGG19

```python
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input
from sklearn.decomposition import LatentDirichletAllocation as LDA
from sklearn.model_selection import train_test_split

# 1. Load VGG19 (pretrained, no top layers)
base_model = VGG19(weights='imagenet', include_top=False, pooling='avg', input_shape=(224, 224, 3))
base_model.trainable = False  # Freeze weights

# 2. Extract features from images (X should be shape: (n_samples, 224, 224, 3))
X_features = base_model.predict(preprocess_input(X))
print(f"VGG19 features shape: {X_features.shape}")  # e.g., (n_samples, 512)
lda = LDA(n_components=10, random_state=42)
X_lda = lda.fit_transform(X_features)  # Reduced to (n_samples, 10)
print("LDA topics learned:", lda.components_.shape)
print("Sample LDA features:", X_lda[:3])
```

Sample Code for QDA+VGG19

```
# 2. Extract features (X: images (n,224,224,3), y: labels)
X_features = base_model.predict(preprocess_input(X))
print(f"VGG19 features: {X_features.shape}")  # (n_samples, 512)

# 3. Split data
X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.2,
random_state=42)

# 4. Train QDA classifier
qda = QDA()
qda.fit(X_train, y_train)

# 5. Predict & evaluate
y_pred = qda.predict(X_test)
print(f"QDA Accuracy: {accuracy_score(y_ ↓ :, y_pred):.3f}")
```

Sample Code for SVM+VGG19

```
# VGG19 features
model = VGG19(weights='imagenet', include_top=False, pooling='avg')
X_feat = model.predict(preprocess_input(X))  # X: your images

# SVM
X_tr, X_te, y_tr, y_te = train_test_split(X_feat, y, test_size=0.2)
svm = SVC(kernel='rbf')
svm.fit(X_tr, y_tr)
print(f"Accuracy: {accuracy_score(y_te, svm.predict(X_te)):.3f}")
```

Sample Code for ETC+VGG19 (Proposed)

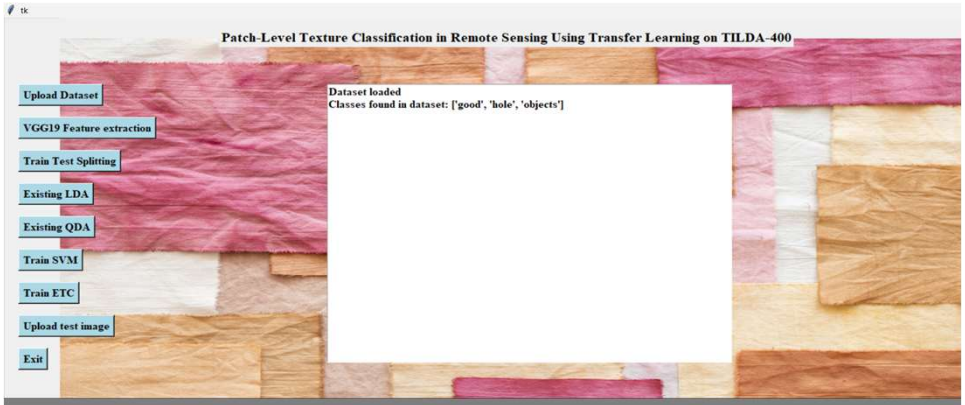
```
# 2. ETC classifier
X_train, X_test, y_train, y_test = train_test_split(X_feat, y, test_size=0.2)
etc = ExtraTreesClassifier(n_estimators=100, random_state=42)
etc.fit(X_train, y_train)

# 3. Results
print(f"ETC Accuracy: {accuracy_score(y_test, etc.predict(X_test)):.3f}")
```

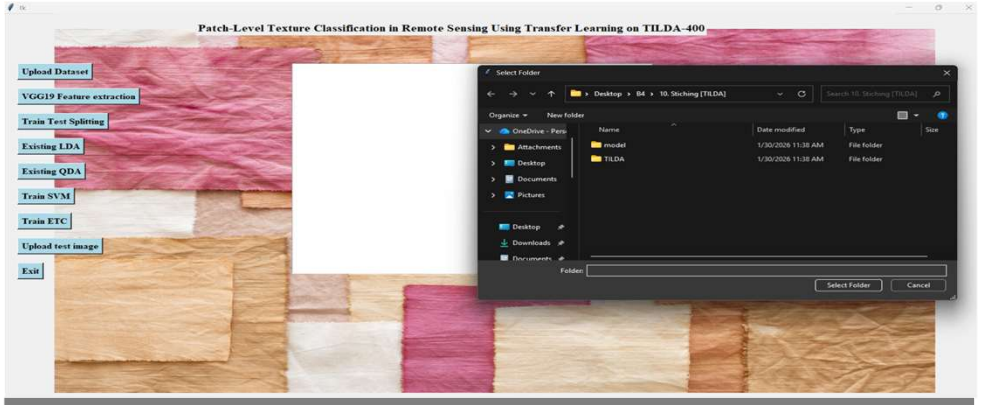


RESULT AND DISCUSSIONS

TKINTER INTERFACE



Classes found



Uploading Dataset

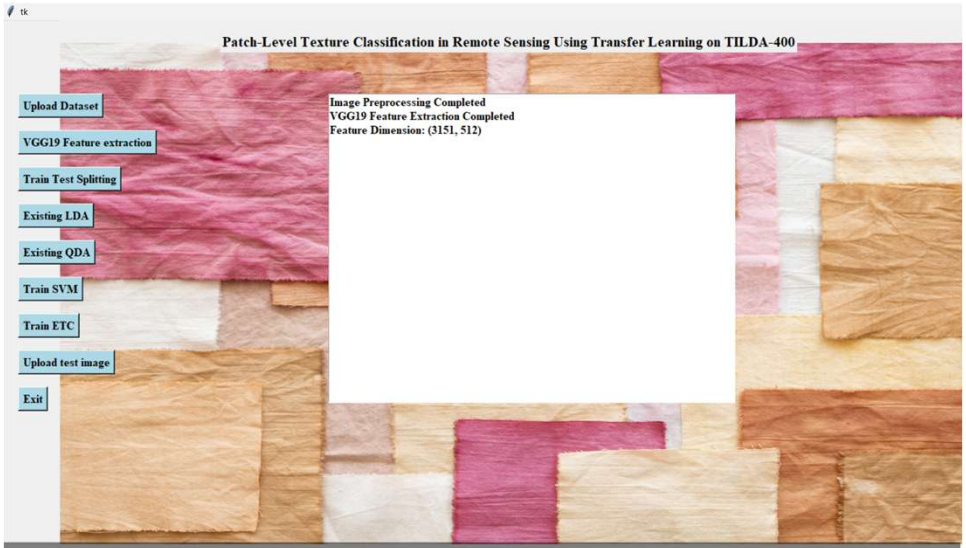
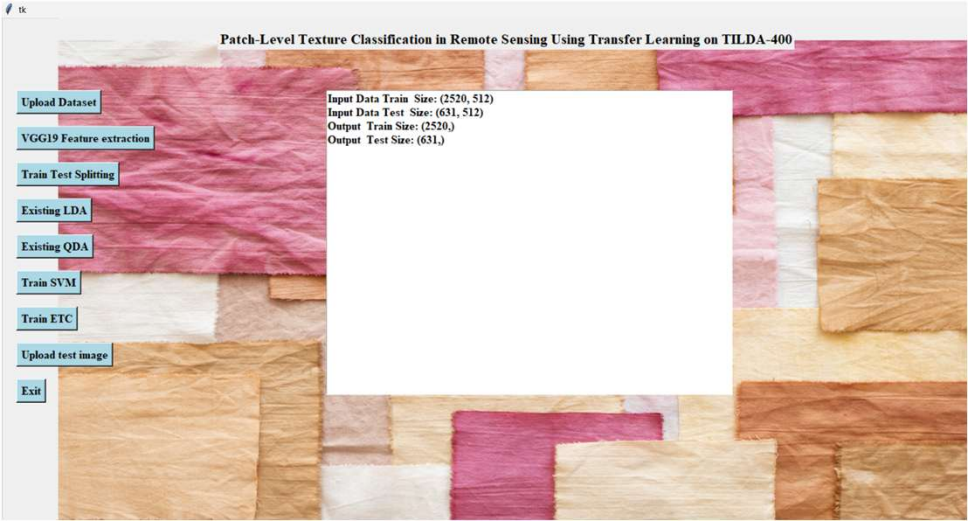
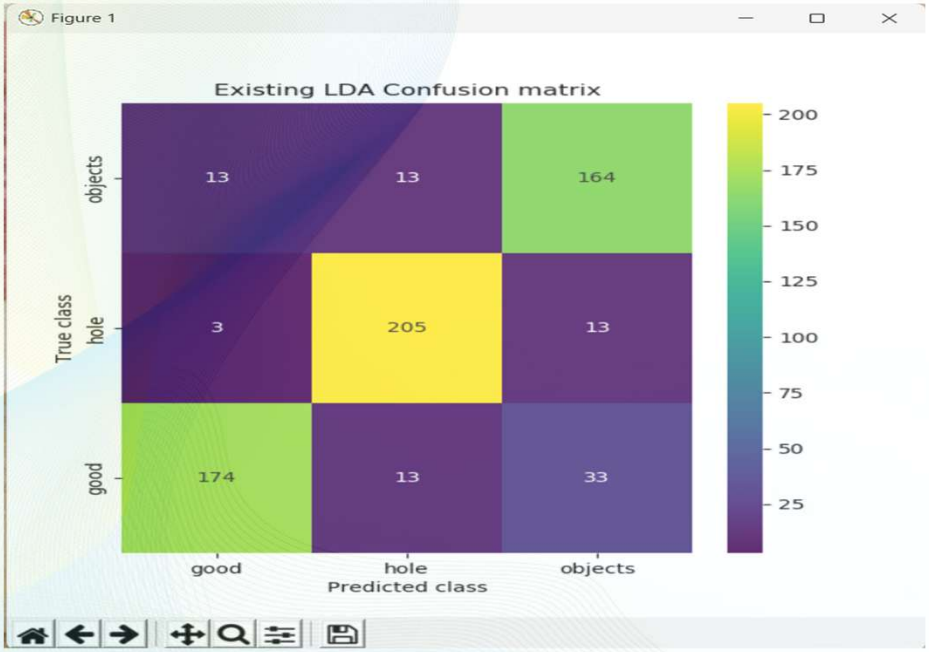


Image Preprocessing



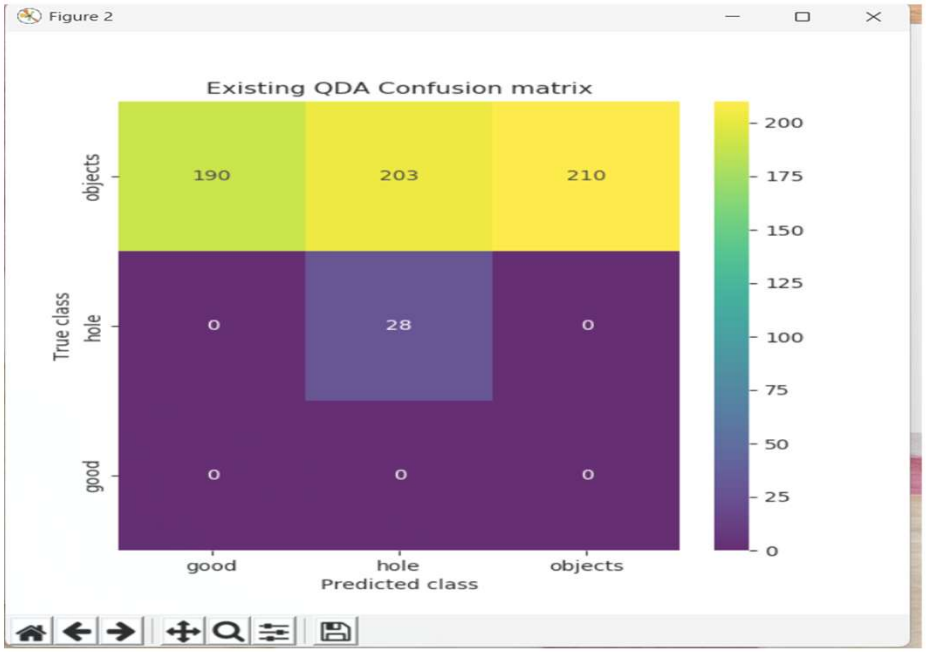
Feature Extraction

Existing LDA model



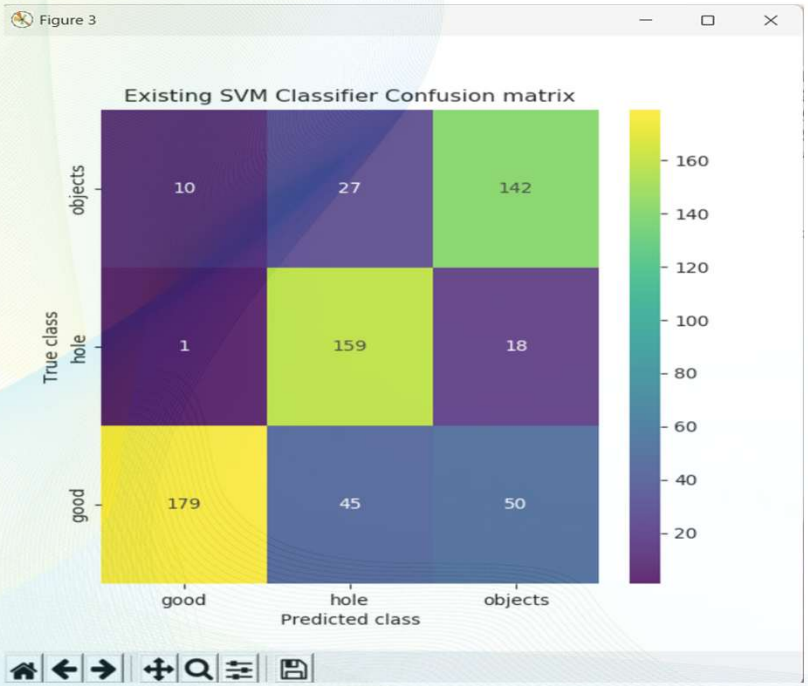
The model shows strong bias toward the "good" class, with very poor detection of "objects" (high misclassification and low recall), indicating class imbalance and weak feature separation. Overall reliability is low for real-world deployment due to unstable object classification performance.

Existing QDA model



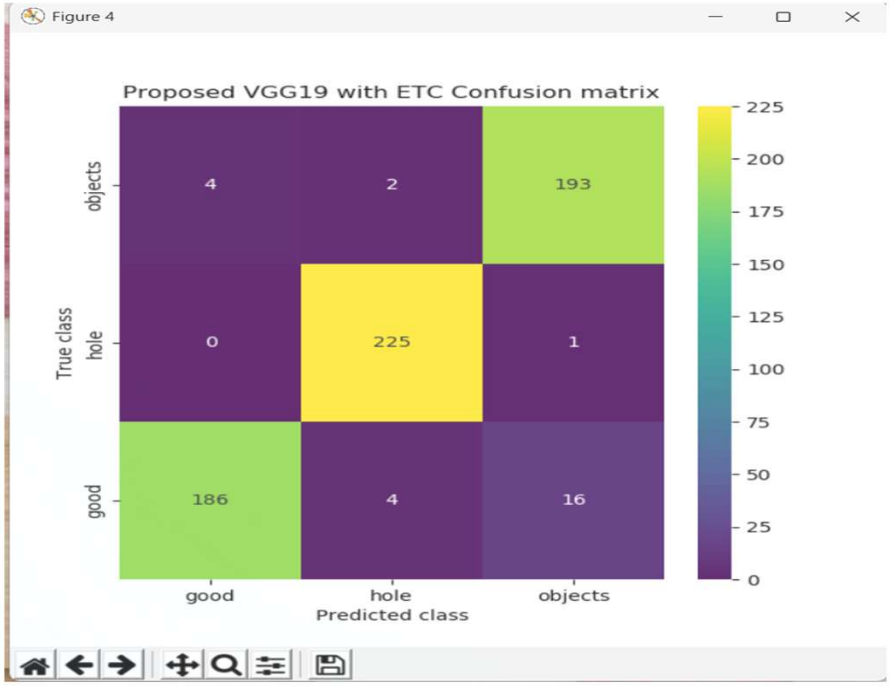
The model is completely biased toward the "objects" class, predicting it 100% of the time, leading to total failure for "good" (0% accuracy) and very poor performance for "hole." The apparent high accuracy for "objects" is misleading, as the model is not learning but simply guessing one class for all inputs.

Existing SVM model



The model performs very poorly on the "objects" class (high misclassification), shows only moderate accuracy for "good," and performs well only on "hole." Overall, performance is inconsistent and unreliable due to weak object detection and significant misclassification rates.

Proposed ETC model



The model achieves excellent and balanced performance across all classes, with near-perfect accuracy for "hole" and very high accuracy for both "good" and "objects." Overall, classification is highly reliable with minimal misclassification and strong class separation.

Existing QDA Accuracy : 37.717908082408876
Existing QDA Precision : 37.37373737373737
Existing QDA Recall : 44.94195688225539
Existing QDA FScore : 24.427379408929227
Existing QDA Sensitivity : nan
Existing QDA Specificity : 100.0

Existing QDA Classification Report

Existing QDA	precision	recall	f1-score	support
good	0.00	0.00	0.00	0
hole	0.12	1.00	0.22	28
objects	1.00	0.35	0.52	603
accuracy			0.38	631
macro avg	0.37	0.45	0.24	631

Existing LDA Accuracy : 86.05388272583201
Existing LDA Precision : 86.1395914027493
Existing LDA Recall : 86.0556265200228
Existing LDA FScore : 85.86200446075259
Existing LDA Sensitivity : 93.04812834224599
Existing LDA Specificity : 98.5576923076923

Existing LDA Classification Report

Existing LDA	precision	recall	f1-score	support
good	0.92	0.79	0.85	220
hole	0.89	0.93	0.91	221
objects	0.78	0.86	0.82	190
accuracy			0.86	631
macro avg	0.86	0.86	0.86	631

Proposed VGG19 with ETC Accuracy : 95.72107765451663
Proposed VGG19 with ETC Precision : 95.73403204982152
Proposed VGG19 with ETC Recall : 95.61123629431057
Proposed VGG19 with ETC FScore : 95.59473113051516
Proposed VGG19 with ETC Sensitivity : 97.89473684210527
Proposed VGG19 with ETC Specificity : 100.0

Proposed VGG19 with ETC Classification Report

Proposed VGG19 with ETC	precision	recall	f1-score	support
good	0.98	0.90	0.94	206
hole	0.97	1.00	0.98	226
objects	0.92	0.97	0.94	199
accuracy			0.96	631

Existing SVM Classifier Accuracy : 76.06973053637084
Existing SVM Classifier Precision : 76.8869142553353
Existing SVM Classifier Recall : 77.99463959615379
Existing SVM Classifier FScore : 75.97116524766636
Existing SVM Classifier Sensitivity : 79.91071428571429
Existing SVM Classifier Specificity : 99.375

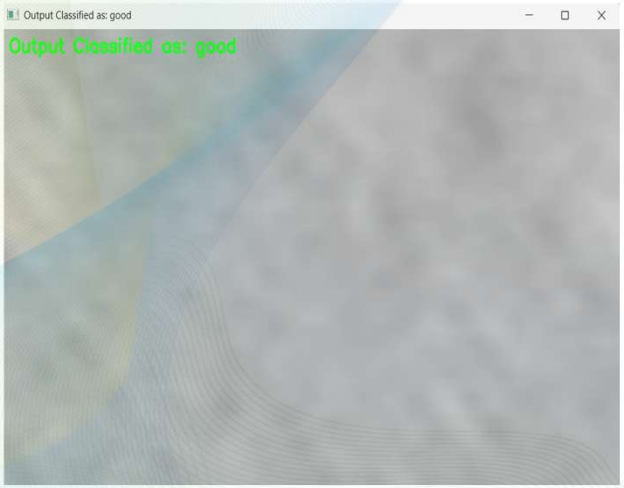
Existing SVM Classifier Classification Report

Existing SVM Classifier	precision	recall	f1-score	support
good	0.94	0.65	0.77	274
hole	0.69	0.89	0.78	178
objects	0.68	0.79	0.73	179
accuracy			0.76	631

Comparative Performance Table of All Models

Metric	LDA	QDA	SVM	ETC (Proposed)
Overall Accuracy	86.05%	37.72%	76.07%	**95.72%**
Precision (Macro Avg)	86.14%	37.37%	77.89%	**95.73%**
Recall (Macro Avg)	86.06%	44.94%	77.99%	**95.61%**
F1-Score (Macro Avg)	85.86%	24.43%	75.97%	**95.59%**
Sensitivity	93.05%	NaN	79.91%	**97.89%**
Specificity	98.56%	100.0%	99.38%	**100.0%**

Output







CORRECTLY CLASSIFIED AS GOOD, OBJECTS AND HOLE

Conclusion

This project presents a transfer learning–based framework for patch- level texture classification using the TILDA-400 dataset. By using deep CNN feature extraction instead of handcrafted features, the system improves accuracy, generalization, and robustness to noise and illumination changes. The Extra Trees Classifier further enhances performance while keeping computation efficient. The framework supports automated training and real-time prediction, making it suitable for practical remote sensing applications like land-cover mapping and environmental monitoring.

Future Scope

- The system can be extended by integrating **Vision Transformer (ViT) and hybrid CNN–Transformer models** to capture long-range spatial dependencies in texture patterns.
- Performance can be further improved by training on **larger and more diverse remote sensing datasets**, including multi-sensor and multi-resolution imagery.
- **Real-time deployment** can be achieved by optimizing the model using lightweight architectures and GPU/edge-based inference.
- The framework can be enhanced to support **unsupervised and semi-supervised learning** for handling rare or unseen texture classes with minimal labeling.
- Integration with **GIS platforms and cloud-based pipelines** can enable large-scale monitoring for applications such as disaster assessment, smart agriculture, and urban planning.

References

1. Dosovitskiy, A. et al., *"An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,"* **Updated surveys & benchmarks**, IEEE Access, 2023.
2. Tan, M. et al., *"Recent Advances in Efficient Deep Learning Models for Image and Texture Classification,"* **ACM Computing Surveys**, 2023.
3. Zhang, Y. et al., *"Deep Transfer Learning for Remote Sensing Texture Classification: A Comprehensive Review,"* **Remote Sensing**, vol. 15, 2023.
4. Liu, J. et al., *"Patch-Level Texture Analysis in High-Resolution Remote Sensing Images Using Deep Neural Networks,"* **IEEE Transactions on Geoscience and Remote Sensing**, 2024.
5. Wang, H. et al., *"Benchmarking CNN and Transformer Models for Texture Classification in Remote Sensing,"* **Pattern Recognition**, 2024.
6. TILDA-400 Dataset, *"Texture Image Library for Deep Learning Applications,"* **Updated Dataset Documentation**, 2023.